EECS 3311 Software Design

Github: https://github.com/nurjahan-shiah/Software-Design

Winter 2026

Sections Z

Group 11

Nurjahan Ahmed Shiah
Joseph Ajeigbe
Jessica Buentipo
David Ekeng
Iyinoluwa Olu-Alabi

2026-02-28

Sequence diagrams and Class diagram are uploaded in the Github for better visibility

Task 1: Use Case Diagram

Requirement	Diagram	Use Case
Req1	Diagram 1	Register Account, Log In
Req2	Diagram 1	Auto-Generate Manager Account
Req3	Diagram 2	Reserve Equipment, Browse Equipment
Req4	Diagram 2	Pay Deposit, Forfeit Deposit
Req5	Diagram 3	Track Equipment Usage
Req6	Diagram 3	Add/Enable/Disable Equipment
Req7	Diagram 3	Assign Equipment Details
Req8	Diagram 1, 2	Provide ID, Modify/Cancel Booking
Req9	Diagram 2	Extend Reservation
Req10	Diagram 2	Select Payment Method

Actor:

1. User
   1.1 Student
   1.2 Faculty
   1.3 Researcher
   1.4 Guest
2. Lab Manager
3. Head Lab Coordinator

Diagram 1: Registration & Account Management [Req1, Req2, Req8]

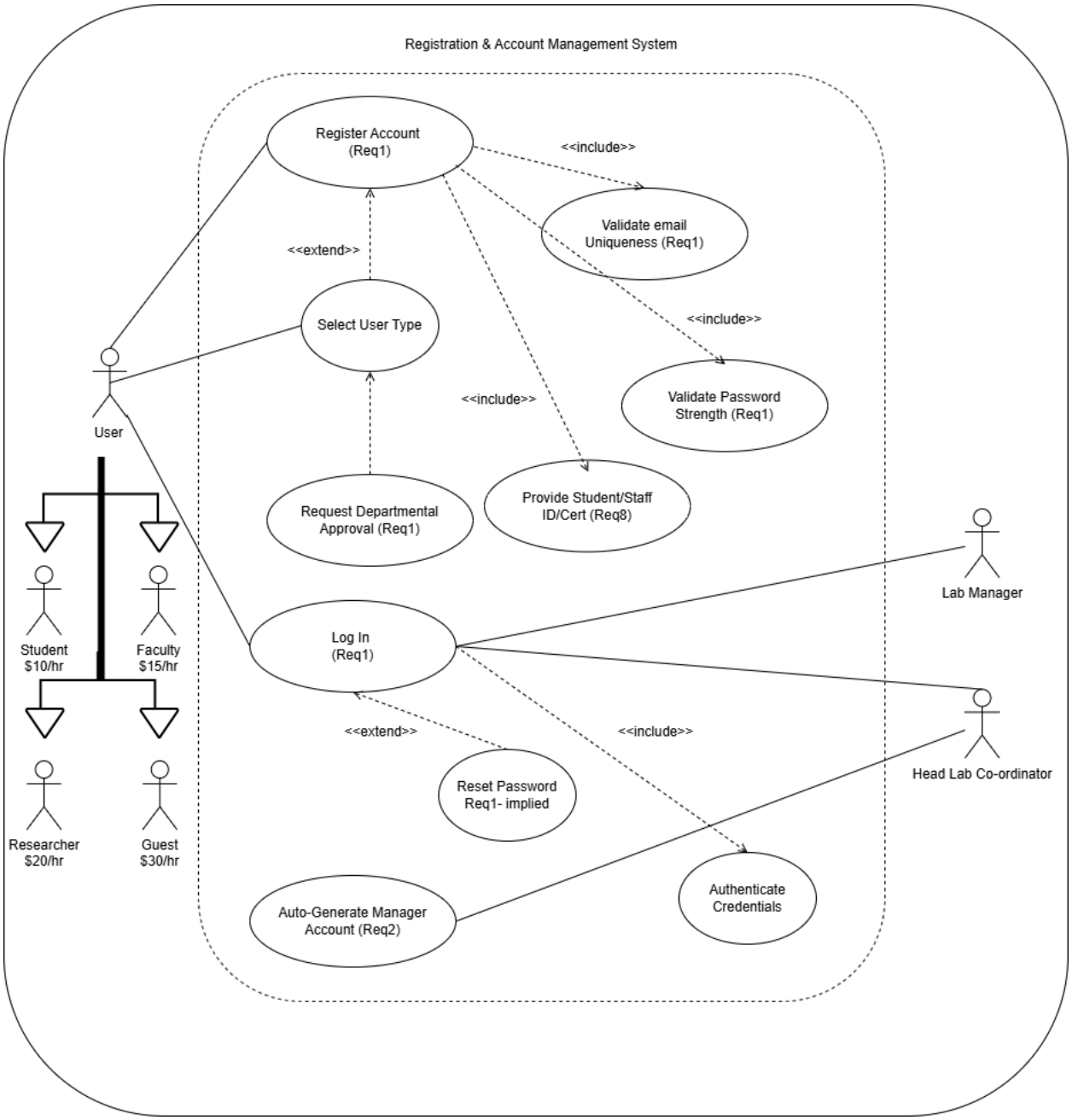


Diagram 2

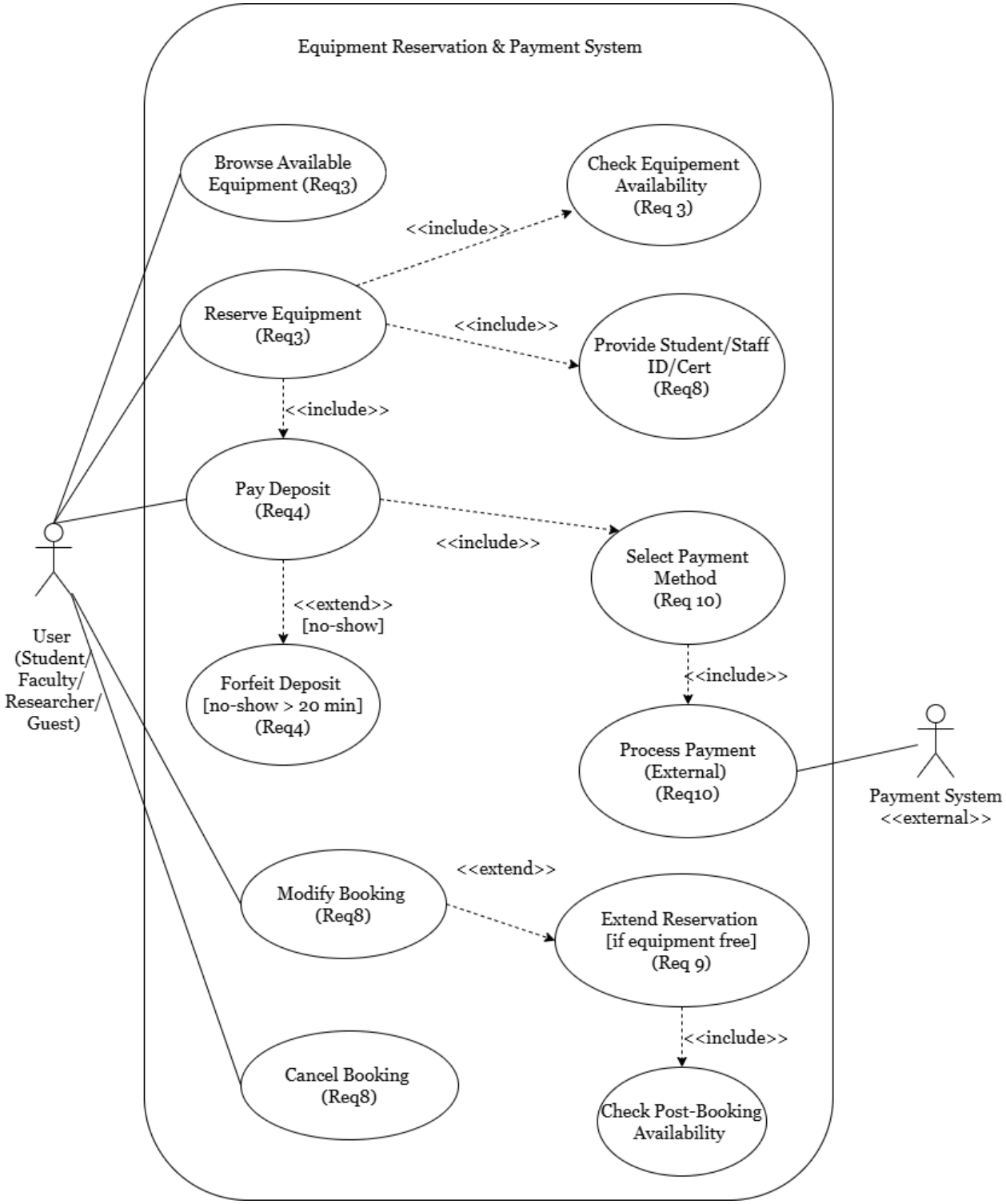
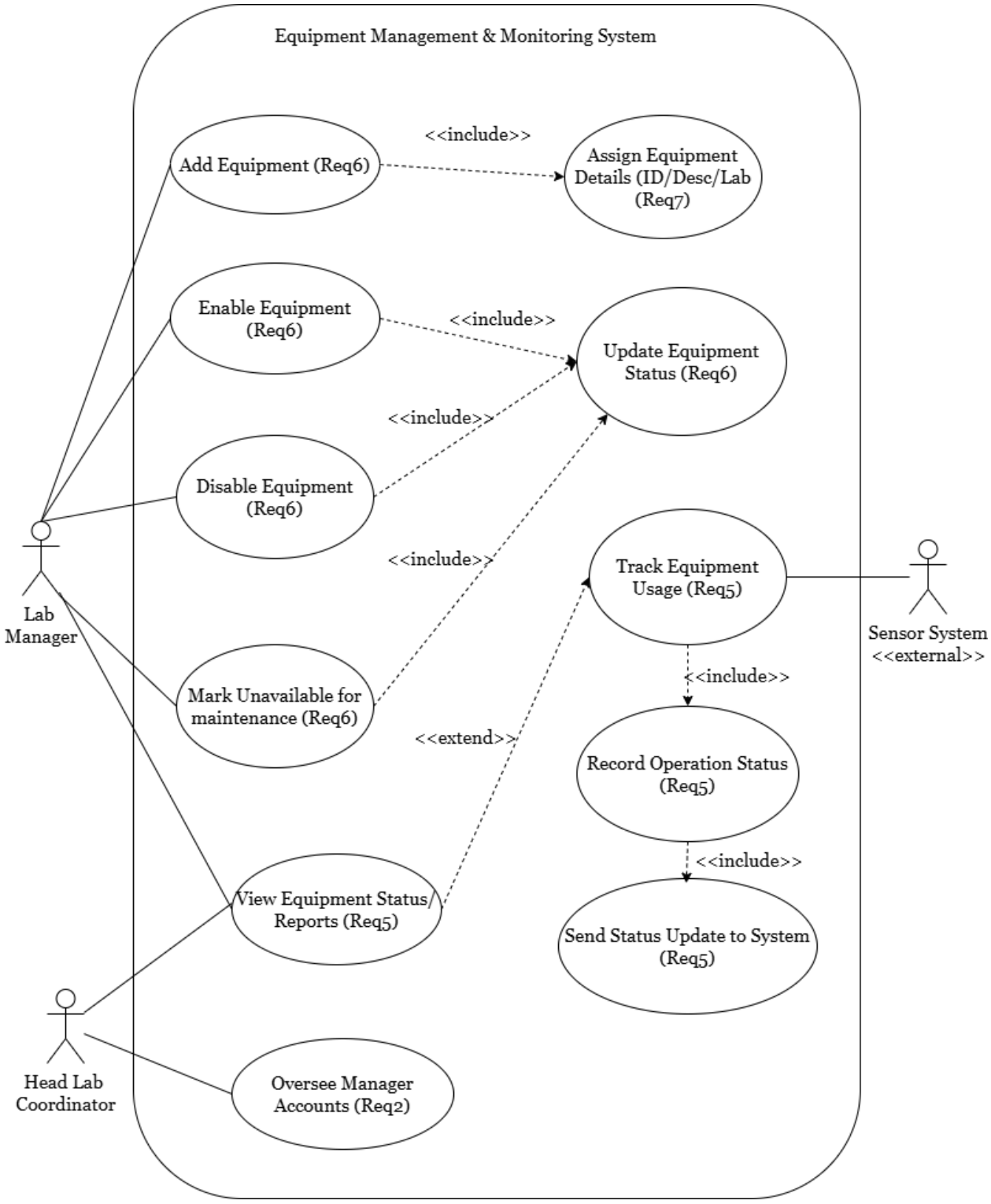
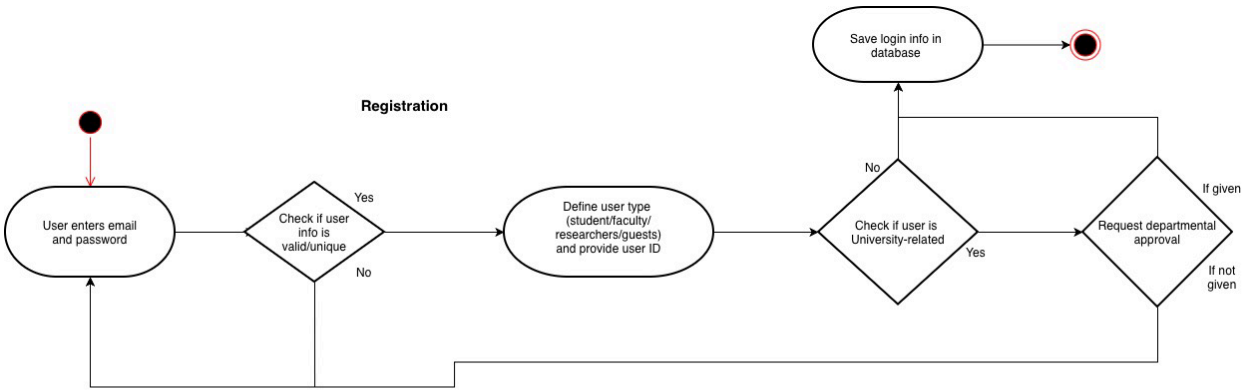

Equipment Reservation & Payment System
Browse Available Equipment (Req3)
Check Equipement Availability (Req 3)
<<include>>
Reserve Equipment (Req3)
<<include>>
Provide Student/Staff ID/Cert (Req8)
<<include>>
Pay Deposit (Req4)
<<include>>
Select Payment Method (Req 10)
<<extend>> [no-show]
Forfeit Deposit [no-show > 20 min] (Req4)
<<include>>
Process Payment (External) (Req10)
Payment System <<external>>
User (Student/ Faculty/ Researcher/ Guest)
<<extend>>
Modify Booking (Req8)
Extend Reservation [if equipment free] (Req 9)
<<include>>
Cancel Booking (Req8)
Check Post-Booking Availability

Diagram 3


Equipment Management & Monitoring System
Add Equipment (Req6)
<<include>>
Assign Equipment Details (ID/Desc/Lab (Req7)
Enable Equipment (Req6)
<<include>>
Update Equipment Status (Req6)
<<include>>
Disable Equipment (Req6)
<<include>>
Track Equipment Usage (Req5)
Sensor System <<external>>
Lab Manager
Mark Unavailable for maintenance (Req6)
<<extend>>
<<include>>
Record Operation Status (Req5)
<<include>>
View Equipment Status/ Reports (Req5)
Send Status Update to System (Req5)
Head Lab Coordinator
Oversee Manager Accounts (Req2)

Task 2: Activity Diagram

Diagram 1



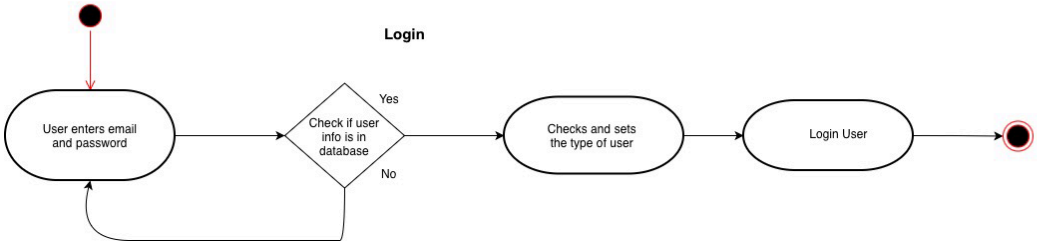
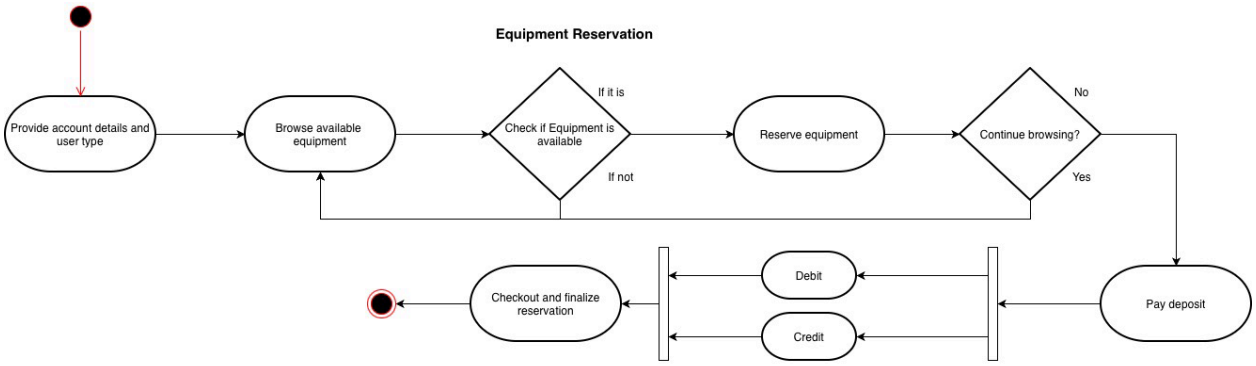


Diagram 2



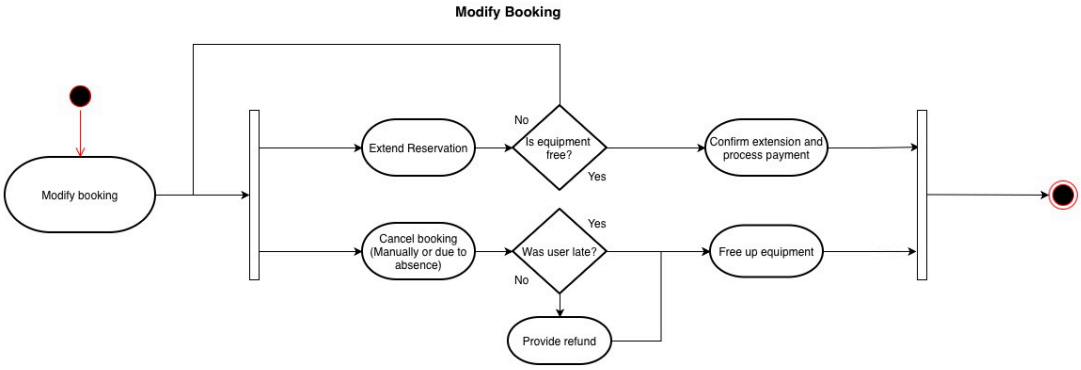
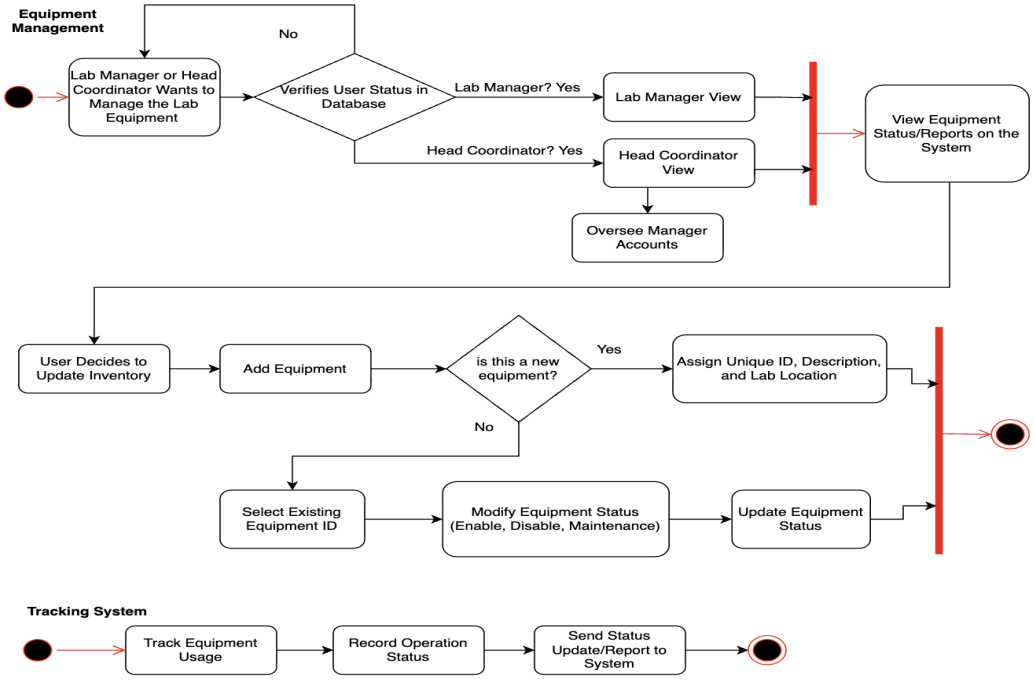


Diagram 3


Equipment Management
No
Lab Manager or Head Coordinator Wants to Manage the Lab Equipment
Verifies User Status in Database
Lab Manager? Yes
Lab Manager View
Head Coordinator? Yes
Head Coordinator View
Oversee Manager Accounts
View Equipment Status/Reports on the System
User Decides to Update Inventory
Add Equipment
is this a new equipment?
Yes
Assign Unique ID, Description, and Lab Location
No
Select Existing Equipment ID
Modify Equipment Status (Enable, Disable, Maintenance)
Update Equipment Status

Tracking System
Track Equipment Usage
Record Operation Status
Send Status Update/Report to System

Task 3: Sequence Diagram

Diagram 1: Registration & Account Management

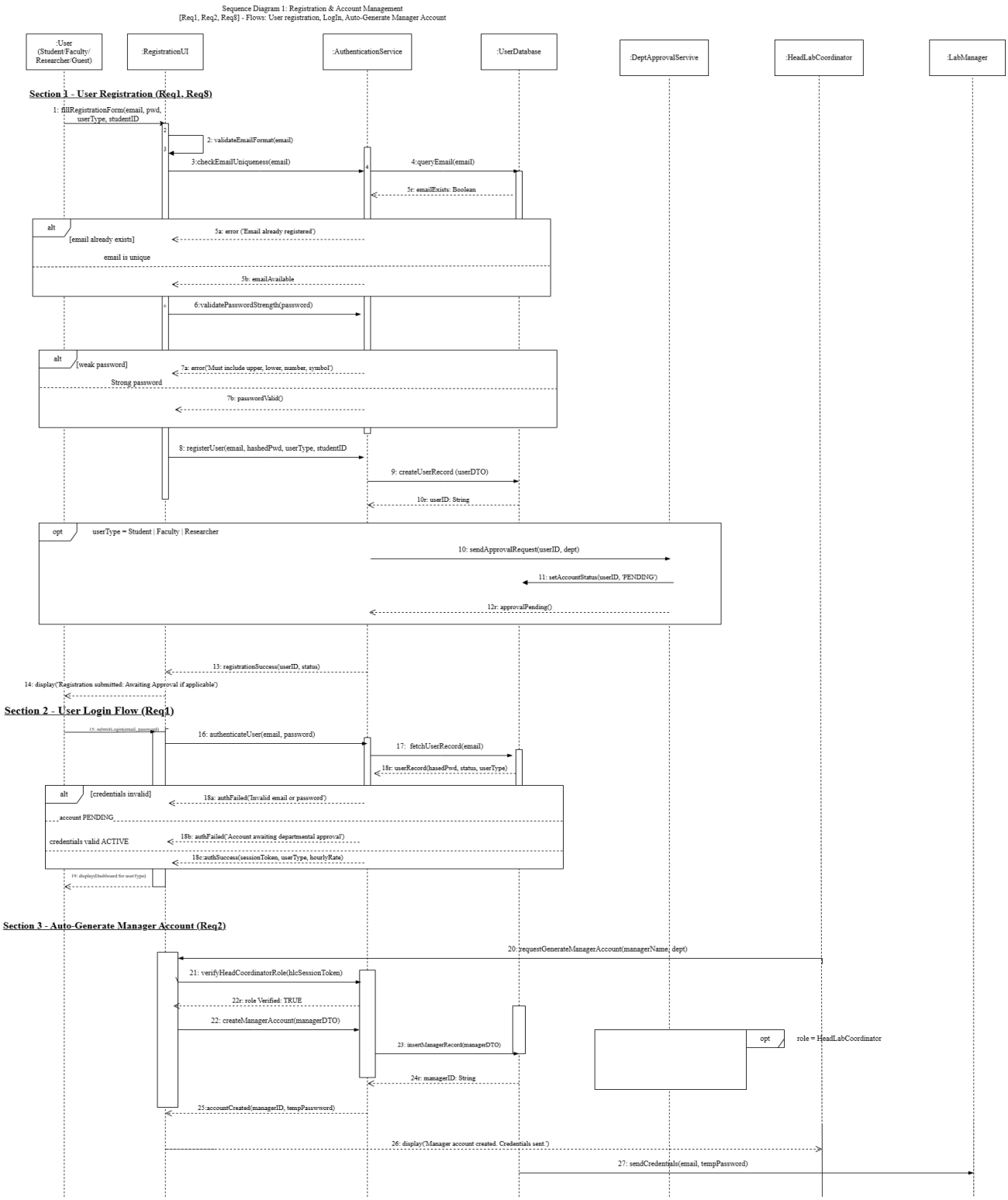


Diagram 2: Equipment Reservation & Payment

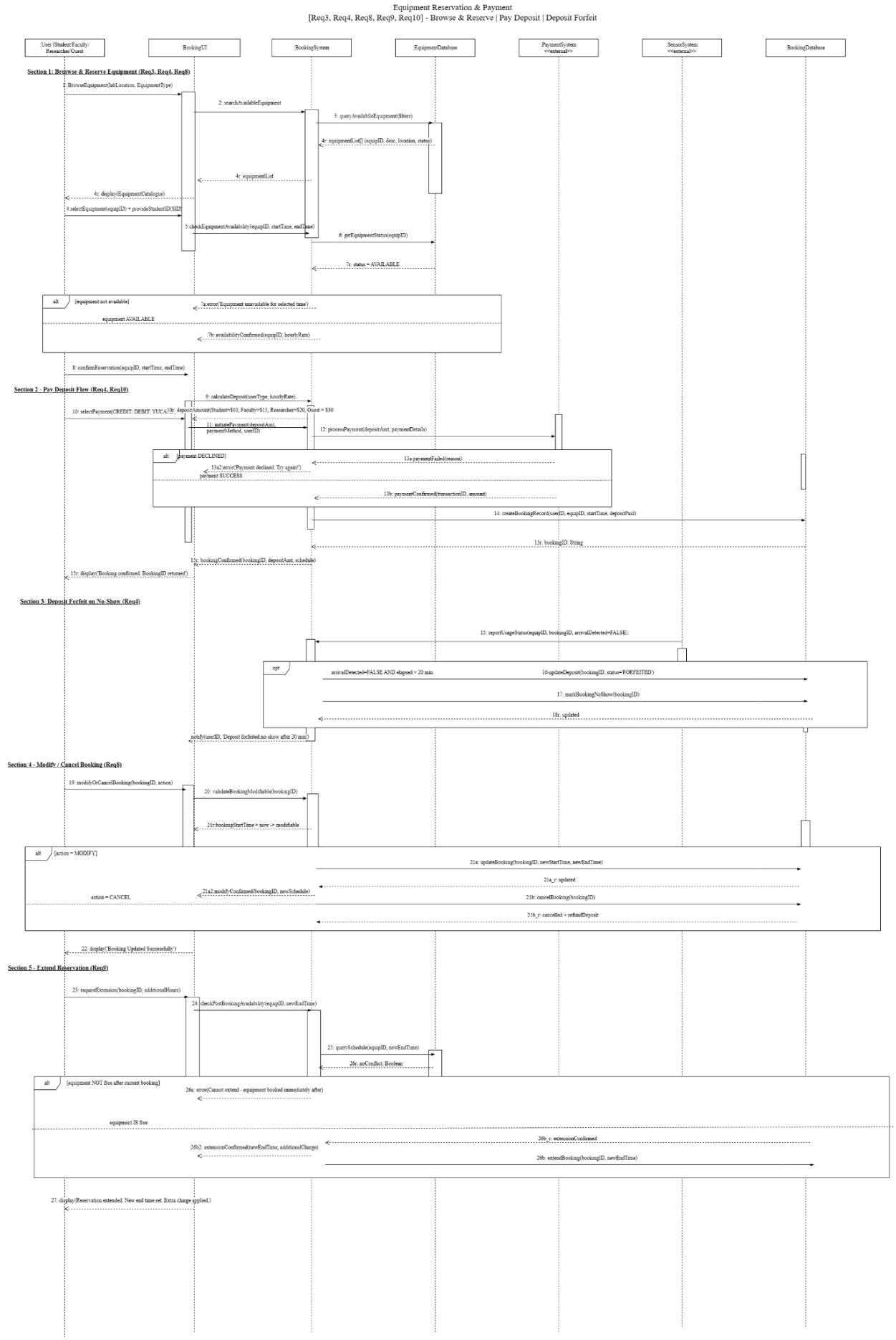
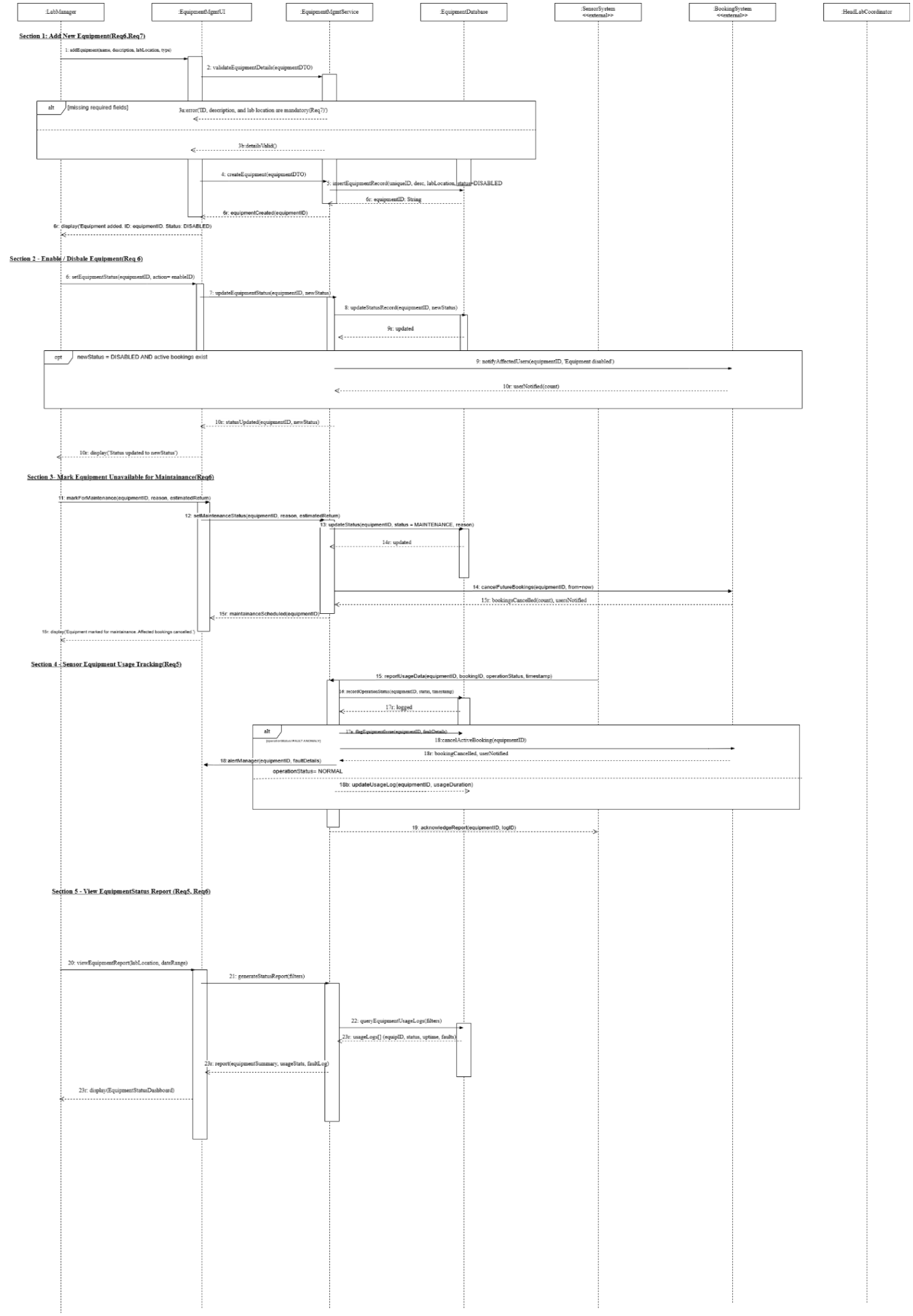


Diagram 3: Equipment Management & Monitoring
[Req5, Req6, Req7] - Flows: Add Equipment, Mark Maintenance, Sensor Status Update



Task 1: Class Diagram

1. User Hierarchy

User (Abstract Class)

- Attributes: - email: String, - password: String, - uniqueID: String, - isApproved: boolean
- Methods: + login(email: String, pass: String): boolean, + register(): void, + requestDepartmentApproval(): void, + getFeeRate(): double
- Relationships: Base class for all specific user types via Inheritance.

Subclasses:

- Student: - hourlyFee: double = 10.0 | + getFeeRate(): double
- Faculty: - hourlyFee: double = 15.0 | + getFeeRate(): double
- Researcher: - hourlyFee: double = 20.0 | + getFeeRate(): double
- Guest: - hourlyFee: double = 30.0, - certificationNumber: String | + getFeeRate(): double

2. Equipment Related

Equipment

- Attributes: - equipmentID: String, - name: String, - description: String, - type: String, - status: String
- Methods: + getStatus(): String, + setStatus(newStatus: String): void
- Relationships: Associated with LabLocation ("located at") and Sensor ("monitored by").

Sensor

- Attributes: - sensorID: String, - linkedEquipmentID: String
- Methods: + trackUsage(bookingID: String): UsageLog
- Relationships: Associated with Equipment; Dependency on EquipmentMgmtService ("sends data").

LabLocation

- Attributes: - buildingName: String, - roomNumber: String
- Methods: + getLocationDetails(): String

3. Booking & Payment

Reservation

- Attributes: - bookingID: String, - startTime: LocalDateTime, - endTime: LocalDateTime, - isExtended: boolean

- Methods: + modifyBooking(newStart: LocalDateTime, newEnd: LocalDateTime): boolean, + cancelBooking(): void, + extendReservation(extraHours: int): boolean
- Relationships: Associated with User ("made by") and Equipment ("reserves"). Contains Deposit via Composition.

Deposit

- Attributes: - amount: double, - isForfeited: boolean, - deductedFromTotal: boolean
- Methods: + validateArrival(arrivalTime: LocalDateTime): void

PaymentTransaction

- Attributes: - transactionID: String, - totalAmount: double, - paymentStatus: String
- Methods: + processFinalPayment(deposit: Deposit): boolean
- Relationships: Associated with Reservation; uses PaymentMethod.

PaymentMethod (Abstract)

- Subclasses: InstitutionalAccount, ResearchGrant, CreditCard, DebitCard.
- Methods: + authorizePayment(amount: double): boolean

4. System Components (Controllers & Services)

BookingSystem

- Attributes: - activeBookings: List<Reservation>
- Methods: + checkActiveBooking(id: String): boolean, + cancelFutureBookings(id: String): void
- Relationships: Manages Reservation; Depends on EquipmentDatabase.

EquipmentMgmtService

- Attributes: - dbConnection: EquipmentDatabase
- Methods: + createEquipment(dto: EquipmentDTO): void, + setEquipmentStatus(id: String, action: String): void, + generateStatusReport(...)
- Relationships: Depends on BookingSystem and EquipmentDatabase.

EquipmentDatabase

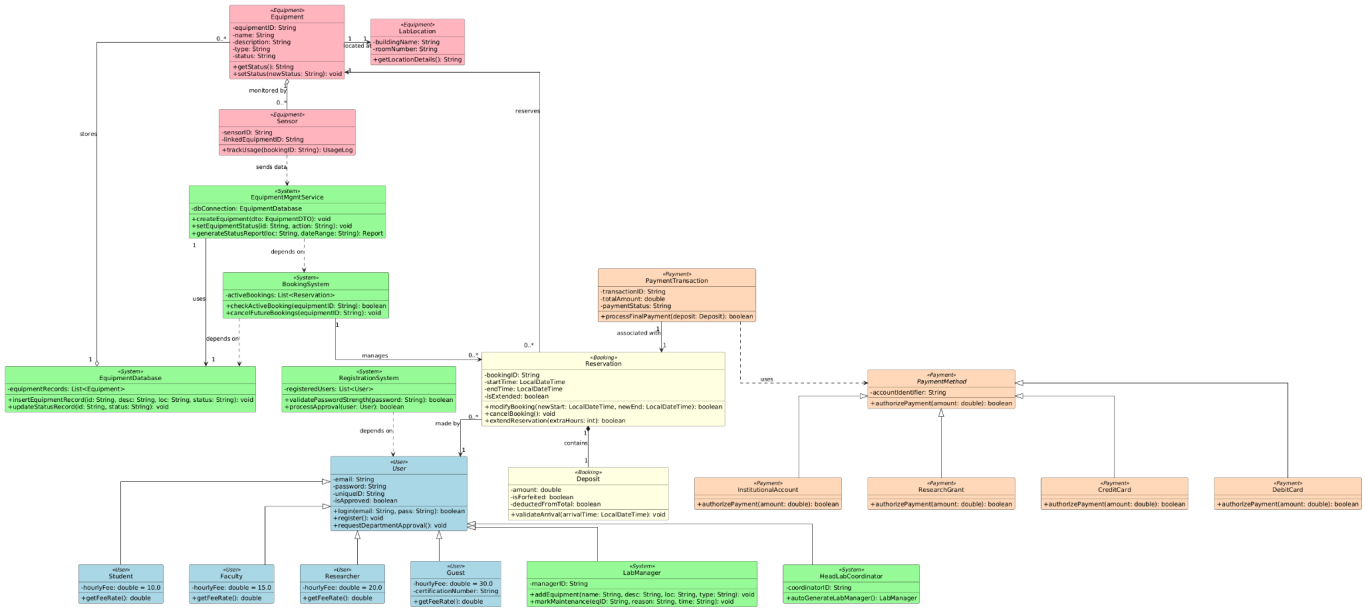
- Attributes: - equipmentRecords: List<Equipment>
- Methods: + insertEquipmentRecord(...), + updateStatusRecord(...)
- Relationships: Stores (Aggregation) Equipment.

RegistrationSystem

- Attributes: - registeredUsers: List<User>
- Methods: + validatePasswordStrength(...), + processApproval(...)
- Relationships: Depends on User.

LabManager & HeadLabCoordinator

- LabManager: + addEquipment(...), + markMaintenance(...)
- HeadLabCoordinator: + autoGenerateLabManager(): LabManager



Part III - AI Assistant Report

Task 1

Task: Use Case Diagram 1

AI Tool: ChatGPT(5.2 Instant) & Claude (Sonnet 4.6)

Prompt: We are designing a UML use case diagram for the registration and authentication portion of a digital booking system for a laboratory equipment reservations system. The system has different user types, including student, faculty, researcher, and guest. It is given that only a head lab coordinator(one person) can create lab manager accounts. Users must validate by providing their student/staff ID or relevant certification number. Bookings can be modified or cancelled before the start.

Output: The AI suggested actors such as User, Administrator, and Authentication Service. It listed use cases including Register Account, Login, Verify Email, and Create User Account. The response put all administrative behaviour under a generic administrator role and placed most validation inside a single “Register” action. It also described authentication as part of the login process and not as a separate system interaction. The explanation mentioned that some actions “may occur optionally” but did not clearly distinguish which should be modelled using <<include>> versus <<extend>> relationships.
Our Understanding: The AI response helped, but it was too broad and generic. It suggested that we split students, faculty, researchers, and guests into different actors; we decided not to do that because their behaviour during registration is the same, and it would just clutter the diagram. We kept one User actor and handled the differences later in the system logic. This made the diagram cleaner.

It also suggested using a general administrator role, but we changed it to a specific Head Lab Coordinator since only that person can generate manager accounts. We also realized the AI kept all verification inside the register step, but from a design perspective, those checks always happen, so we separated them and used <<include>> for email uniqueness, password strength, and ID verification.

Our Understanding: The AI response helped, but it was too broad and generic. It also suggested that we introduce one actor (user) but we split students, faculty, researchers, and guests into different actors. We decided to do that even though their behaviour during registration is the same but the pay is different.

Task: Use Case Diagram 2

AI Tool: ChatGPT (5.2 Instant) & Claude (Sonnet 4.6)

Prompt: We are designing a UML use-case diagram for equipment reservation and payment in a lab booking system. Requirements: different hourly rates depending on the user type, one-hour deposit required and forfeited if the user is more than 20 minutes late, users are able to extend the reservation if the equipment is free, and there is support for multiple payment methods (credit/debit/institutional grants). Suggest actors and use cases, and indicate include/extend relations.

Output: The AI listed actions like Reserve Equipment, Make Payment, and Cancel Booking. It mentioned deposits and availability checks, but did not clearly show the 20 minute forfeiture condition or separate mandatory steps from optional ones. Payment was described as a single action, and the connections between steps were not shown with UML notation.

Our understanding: We updated the workflow in the UML by including Pay Deposit as part of the reservation process and setting Forfeit Deposit as an extension that applies if someone is a no-show for more than 20 minutes. We also separated payment selection from external payment processing and added mandatory ID verification for booking.

Task: Use Case Diagram 3

AI Tool: ChatGPT (5.2 Instant) & Claude (Sonnet 4.6)

Prompt: We are designing a UML use case diagram for the equipment management and monitoring portion of a lab booking system. A lab manager can add, enable, disable, or mark equipment as unavailable for maintenance. Each equipment item has a unique ID, description, and lab location. Sensors automatically track equipment usage and send status updates to the system. Suggest actors, use cases, and relationships.

Output: The AI suggested actors including Lab Manager and System Administrator. It listed use cases such as Add Equipment, Update Equipment, and Monitor Equipment. It grouped all sensor-related behaviour under a generic "System" actor and treated equipment tracking as an internal process rather than an external actor interaction. It also did not separate Enable, Disable, and Mark for Maintenance as distinct use cases, instead combining them into a single "Update Equipment Status" action. The AI did not mention the Head Lab Coordinator as a separate actor or include the Oversee Manager Accounts use case.

Our Understanding: The AI response gave us a starting point but missed several important distinctions. We separated Enable Equipment, Disable Equipment, and Mark Unavailable for Maintenance as individual use cases because each represents a distinct manager action with different system consequences, for example, marking for maintenance also triggers booking cancellations, which enabling/disabling does not. We also added the Sensor System as an external actor rather than modelling sensor updates as internal system behaviour, which better reflects the UML definition of an actor as something that interacts with the system from outside.

Finally, we added Head Lab Coordinator as a separate actor with the Oversee Manager Accounts use case, since Req2 specifies this role has unique responsibilities not shared with the Lab Manager.

Task: Activity Diagram 1

AI Tool: Claude (Sonnet 4.6)

Prompt: When modelling an activity diagram for a booking system, should actions like checking equipment availability and processing payment be sequential or can they occur in parallel, and how should that be shown in UML?

Output: The AI explained that actions in an activity diagram should only run in parallel if they can happen independently. If one action relies on the result of another, they should be shown in sequence using a normal control flow. For example, payment should not happen at the same time as checking availability if the booking depends on availability. In that case, it is better to use a simple flow instead of a fork or join.

Our Understanding: For Activity Diagram 1 (Registration), we applied this principle to the relationship between email/password validation and the departmental approval step. We initially considered whether both validation checks, email uniqueness and password strength, could run in parallel using a fork node, since they check independent fields. However, we decided against this because in practice the system would need both results before proceeding to the next step, and the diagram would become unnecessarily complex for what is effectively a simple sequential gate. We kept them as sequential decision nodes. The departmental approval branch only activates after the user type is confirmed as university-affiliated, which is a data dependency, approval cannot be requested until the type is known. The login sub-flow was kept entirely sequential for the same reason: the system cannot set the user type until the database confirms the credentials are valid. The AI's guidance confirmed that sequential flow was correct here and that adding fork/join nodes would misrepresent the actual control flow.

Task : Activity Diagram 2

AI Tool: Claude (Sonnet 4.6)

Prompt: "When modelling an activity diagram for a booking system, should actions like checking equipment availability and processing payment be sequential or can they occur in parallel, and how should that be shown in UML?"

Output: The AI explained that actions in an activity diagram should only run in parallel if they can happen independently. If one action relies on the result of another, they should be shown in sequence using a normal control flow. For example, payment should not happen at the same time as checking availability if the booking depends on availability. In that case, it is better to use a simple flow instead of a fork or join.

Our Understanding: Activity Diagram 2 (Equipment Reservation) was the diagram most directly addressed by the AI's response. We had originally considered modelling the availability check and deposit payment as parallel branches using a fork node, reasoning that the system could begin preparing the payment interface while confirming availability in the background. After reading the

AI's explanation, we recognized this would be incorrect because the deposit amount depends on the user type and the hourly rate, and the payment step should only be reachable if availability is confirmed. Initiating payment before availability is confirmed would allow the user to pay a deposit for equipment that turns out to be unavailable, which contradicts Req3 and Req4. We therefore kept the flow strictly sequential: browse equipment → check availability → reserve → pay deposit → select payment method → checkout. The modified booking sub-flow includes a fork node to show that extending and cancelling are parallel branches of the same decision, since only one will execute depending on the user's choice , this is a decision node rather than a true parallel fork, which the AI's output helped us clarify. The AI response was the primary reason we kept payment downstream of availability confirmation rather than alongside it.

Task : Activity Diagram 3

AI Tool: Claude (Sonnet 4.6)

Prompt: "When modelling an activity diagram for a booking system, should actions like checking equipment availability and processing payment be sequential or can they occur in parallel, and how should that be shown in UML?"

Output: The AI explained that actions in an activity diagram should only run in parallel if they can happen independently. If one action relies on the result of another, they should be shown in sequence using a normal control flow. For example, payment should not happen at the same time as checking availability if the booking depends on availability. In that case, it is better to use a simple flow instead of a fork or join.

Our Understanding: For Activity Diagram 3 (Equipment Management), the AI's guidance on parallelism influenced how we modelled the three sub-flows, Equipment Management, Inventory Update, and Tracking System. We considered whether these three flows could be shown as parallel branches starting from a single fork node, since they represent different workflows that could technically happen simultaneously in a real system. We decided to keep them as separate independent flows each with their own start node, rather than connecting them with a fork, because they are triggered by completely different actors and events: the Equipment Management flow starts when a Lab Manager or Head Coordinator logs in, the Inventory Update flow starts when a user decides to update inventory, and the Tracking System flow starts automatically when the sensor system activates. Connecting them with a fork would imply they all start from the same trigger, which is inaccurate. Within the Equipment Management flow, we

used a fork node to show that Lab Manager View and Head Coordinator View can branch in parallel depending on the verified user role, since both lead to the same View Equipment Status action, this is the one case where the AI's parallel flow guidance supported using a fork. For the Inventory Update flow, the decision between adding new equipment and modifying existing equipment is a decision node, not a fork, since only one path executes, consistent with the AI's guidance that parallel nodes should only be used when both branches genuinely occur simultaneously.

Task: Sequence Diagram 1

AI Tool: Claude (Sonnet 4.6)

Prompt: "We are creating a UML sequence diagram for the registration and account management scenario of a lab equipment reservation system. The scenario includes user registration with email and password validation, departmental approval for university-affiliated users, user login with authentication, and the head lab coordinator auto-generating manager accounts. What lifelines, messages, and conditional flows should be included?"

Output: The AI suggested three lifelines: User, System, and Database. It showed a basic linear flow of registration where the user submits a form, the system checks the email, and the database stores the record. It included login as a single message from User to System returning a success or failure. It did not separate the AuthenticationService, DeptApprovalService, or RegistrationUI as distinct lifelines. The auto-generated manager account flow was not included at all. The AI mentioned that validation could be shown with "if/else comments" beside arrows rather than using formal combined fragments. Return messages were not included.

Our Understanding: The AI's output was too simplified. We restructured the diagram significantly. We separated RegistrationUI, AuthenticationService, UserDatabase, and DeptApprovalService into distinct lifelines because each represents a separate system component with its own responsibilities , collapsing them into a single "System" lifeline would prevent a developer from understanding how the system is internally structured. We replaced the AI's informal if/else comments with proper alt combined fragments for email uniqueness and password strength checks, since both have two distinct branches (error vs. success) that must be shown formally. We used an opt fragment for the departmental approval step since it only applies to Student, Faculty, and Researcher user types and has no alternative branch. We added Section 2 for the Login flow with its own alt fragment covering three cases: invalid credentials, pending account, and successful login , none of which the AI had modelled. We added Section 3 for the Auto-Generate Manager Account flow entirely from scratch since the AI omitted it, even though it directly addresses Req2. Activation bars and dashed return messages were added throughout, as the AI had not included either.

Task: Sequence Diagram 2

AI Tool: Claude (Sonnet 4.6)

Prompt: "We are creating a UML sequence diagram for the equipment reservation and payment scenario of a lab booking system. The scenario includes browsing and reserving equipment, paying a one-hour deposit with support for credit, debit, institutional accounts, and research grants, forfeiting the deposit if the user does not arrive within 20 minutes, modifying or cancelling

a booking before it starts, and extending a reservation if the equipment is free afterward. What lifelines and message flows should be included?"

Output: The AI suggested lifelines for User, BookingSystem, and PaymentGateway. It showed a basic reserve-then-pay flow where the user selects equipment, the system confirms availability, and the payment gateway processes a payment. It described the deposit as a note beside the payment step rather than as a distinct modelled flow. The 20-minute forfeiture condition was mentioned in a comment but not modelled as a sequence flow. Modify, cancel, and extend reservation were listed as bullet points of "possible user actions" without any message flows or lifelines shown for them. The PaymentSystem, SensorSystem, EquipmentDatabase, and BookingDatabase were not included as separate lifelines.

Our Understanding: The AI response captured the general idea but missed most of the required detail. We expanded the diagram into 5 distinct sections across 2 pages. Section 1 covers Browse and Reserve with an alt fragment for equipment availability. Section 2 covers Pay Deposit with an alt fragment for payment declined versus success, showing all four payment method types in the message label as required by Req10. Section 3 covers Deposit Forfeit as a completely separate flow initiated by the SensorSystem reporting no arrival , the AI had not modelled this at all, but Req4 explicitly requires it. We used an opt fragment with the guard [arrivalDetected=FALSE AND elapsed > 20min] to show it only triggers under that condition.

Section 4 covers Modify and Cancel Booking with an alt fragment separating the two actions and their different database outcomes. Section 5 covers Extend Reservation with an alt fragment for equipment free versus not free after the current booking, directly addressing Req9. We added BookingDatabase, EquipmentDatabase, PaymentSystem, and SensorSystem as separate lifelines since the AI had collapsed all system behaviour into a single BookingSystem lifeline, which was insufficient for implementation-level detail.

Task : Sequence Diagram 3

AI Tool: Claude (Sonnet 4.6)

Prompt: "We are creating a UML sequence diagram for the equipment management and sensor monitoring scenario of a lab booking system. The scenario includes a lab manager adding new equipment with mandatory ID, description, and lab location fields, enabling or disabling equipment with notification to affected users, marking equipment as unavailable for maintenance and cancelling future bookings, sensors automatically reporting equipment usage status to the system, and the lab manager viewing equipment status reports. What lifelines, messages, and conditional flows should be modelled?"

Output: The AI suggested three lifelines: LabManager, System, and Database. It showed a basic flow where the manager adds equipment and the system stores it. It combined all management actions , add, enable, disable, maintenance , into a single generic "updateEquipment" message. The sensor tracking flow was described as "the system periodically polls the sensor" rather than modelling the sensor as an actor that pushes data to the system. It did not include BookingSystem as a lifeline, so the interaction between equipment management and active bookings when equipment is disabled or marked for maintenance was not shown. Return messages and activation bars were absent.

Our Understanding: The AI misunderstood the sensor interaction direction, which was the most significant error. Req5 states that sensors send updates to the system , the system does not poll them. We corrected this by having the SensorSystem lifeline initiate the message flow in Section 4, sending reportUsageData() to the EquipmentMgmtService, which is the correct direction. We also separated all management actions into distinct sections. Section 1 covers Add Equipment with an alt fragment for missing required fields, directly enforcing Req7 which mandates that ID, description, and lab location are all mandatory. Section 2 covers Enable and Disable with an opt fragment that only notifies affected users when the new status is DISABLED and active bookings exist, this conditional was entirely absent from the AI's output. Section 3 covers Mark for Maintenance, which includes a call to BookingSystem to cancel future bookings , a consequence the AI did not model at all. Section 4 covers Sensor Tracking with an alt fragment distinguishing between a fault or anomaly status versus a normal operation status, since the system must respond differently in each case per Req5. Section 5 covers View Equipment Status Reports as a separate read-only flow. We added EquipmentMgmtService, EquipmentDatabase, BookingSystem, SensorSystem, and HeadLabCoordinator as separate lifelines since the AI's single "System" lifeline was far too coarse for implementation-level documentation.

Task 2: Disagreements & Resolution

AI mixed up include vs extend semantics . AI used phrases like “may occur optionally” without recommending the correct UML relationship.
Using include vs extend correctly is important for formal UML diagrams. We fixed this by reviewing UML guidelines and explicitly annotating every relationship in our diagrams. We used «include» where a sub-behaviour always happens (validation, payment deposit, updating equipment status) and «extend» where behaviour is conditional (forfeiture, optional user type selection, reservation extension).

AI omitted key requirement details (deposit forfeiture and ID verification). It missed the 20-minute deposit forfeiture rule and the requirement that users provide a student or staff ID or certification for bookings. These are explicit requirements (Req4, Req8) and must be shown either as use cases or preconditions. We fixed it by adding provide Student/Staff ID/Cert as a required use case , forfeit Deposit as an <<extend>> of Pay Deposit with the condition of a no-show when greater than 20 min..

AI treated login and authentication as the same step, AI assumed that login and authentication are both one action. But the issue with this is that pressing login is a user action, and authentication is a system verification step. We fixed it by separating them; log In <<include>> Authenticate Credentials
This shows the system checks information after the user action.

AI tried to include UI elements in UML. The AI suggested buttons and screens as actions but use-case diagrams represent behaviour, not interface. We fixed this by making sure to not include any UI-level steps and keeping only system interactions.

Task 3: Verifying AI responses

We did not directly trust the AI responses when creating the diagrams. Instead, we treated them as suggestions and checked them before using them. Most of the time, the AI described the idea correctly, but not in proper UML form, so we had to verify both the requirements and the notation.

We verified the suggestion by:

- Validating against the requirements: After getting a response, we compared it with the listed requirements (Req1–Req10). If the AI missed a condition, we changed the diagram to match the specification.
- Checked against UML 2.5 standards: We made sure the suggestion followed UML conventions. In particular, we checked that actors were external entities interacting with the system, «include» should represent required steps, «extend» should represent conditional behaviour. We also fact-checked with the lecture notes.
- Used UML tools like UMlet.
- - Reviewed all the diagrams as a group and made sure everyone agreed with each decision that was made.

Task 4: Applied AI Content

Prompt: “In a UML use case diagram for a digital booking system for laboratory equipment reservations do sensors automatically send equipment usage information to the system, should that be modeled as a user action or an external actor interaction?”

Response: “If an external device automatically provides information to the system, it should be modeled as an external actor interacting with the system, not as a user action. The system receives the input rather than initiating it.”

We used this directly in the equipment monitoring use case diagram, where the Sensor System is shown as an external actor connected to Track Equipment Usage and related monitoring actions instead of modeling equipment updates as user-triggered behavior.

Rationale for accepting AI’s response: We went with this suggestion because it matched what we wanted to show in the diagram. Since the sensors send data on their own, it made more sense to model them as a separate external actor instead of as a user action. When we checked our lecture notes, we saw that this fit the UML definition of an actor, so we decided to keep it as it was, unlike most other AI suggestions we changed.

Note: We have also instructed AI to demo the use case diagrams which is included below (prompted with chatGPT, later refined through Claude):

Diagram 1:

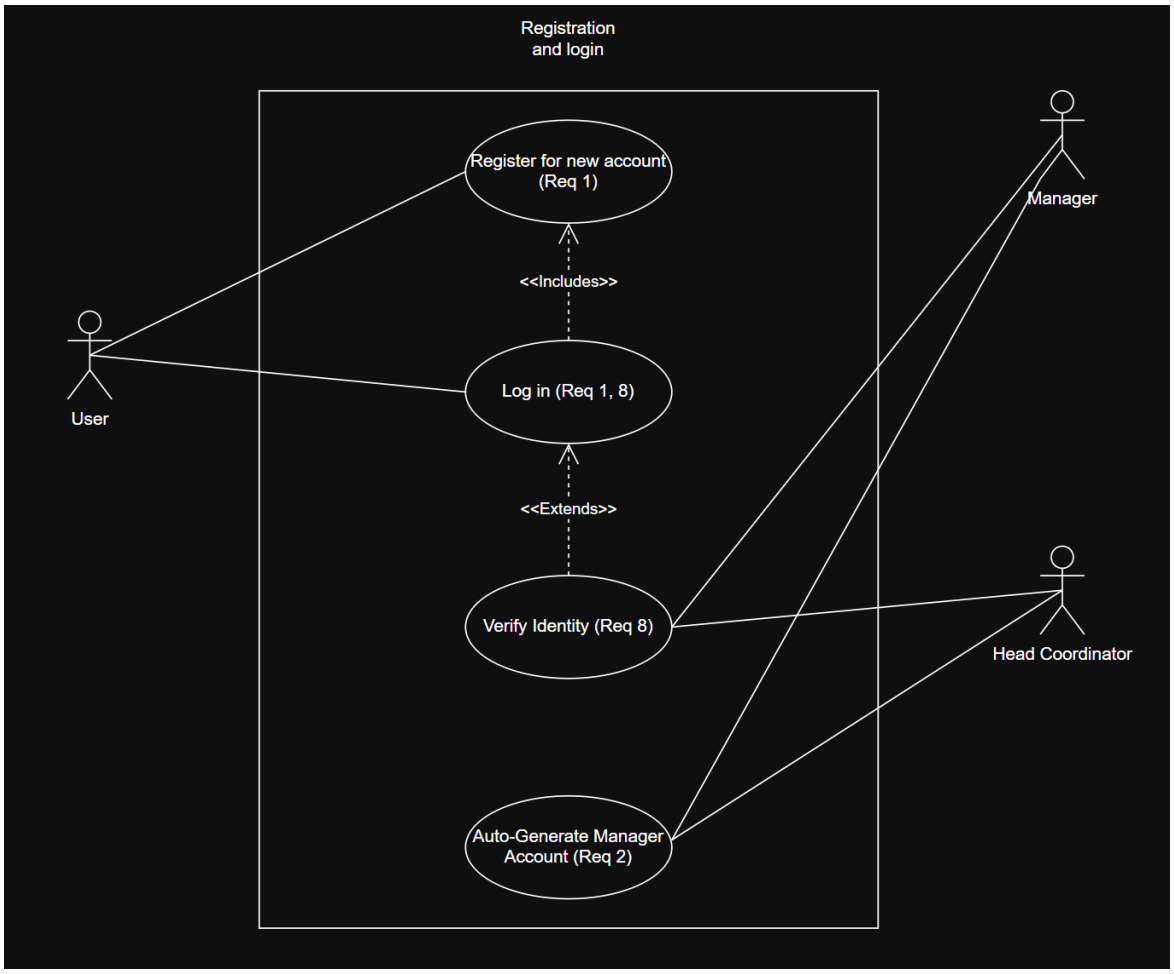

Registration
and login
Register for new account
(Req 1)
<<Includes>>
Log in (Req 1, 8)
<<Extends>>
Verify Identity (Req 8)
Auto-Generate Manager
Account (Req 2)
User
Manager
Head Coordinator

Diagram 2:

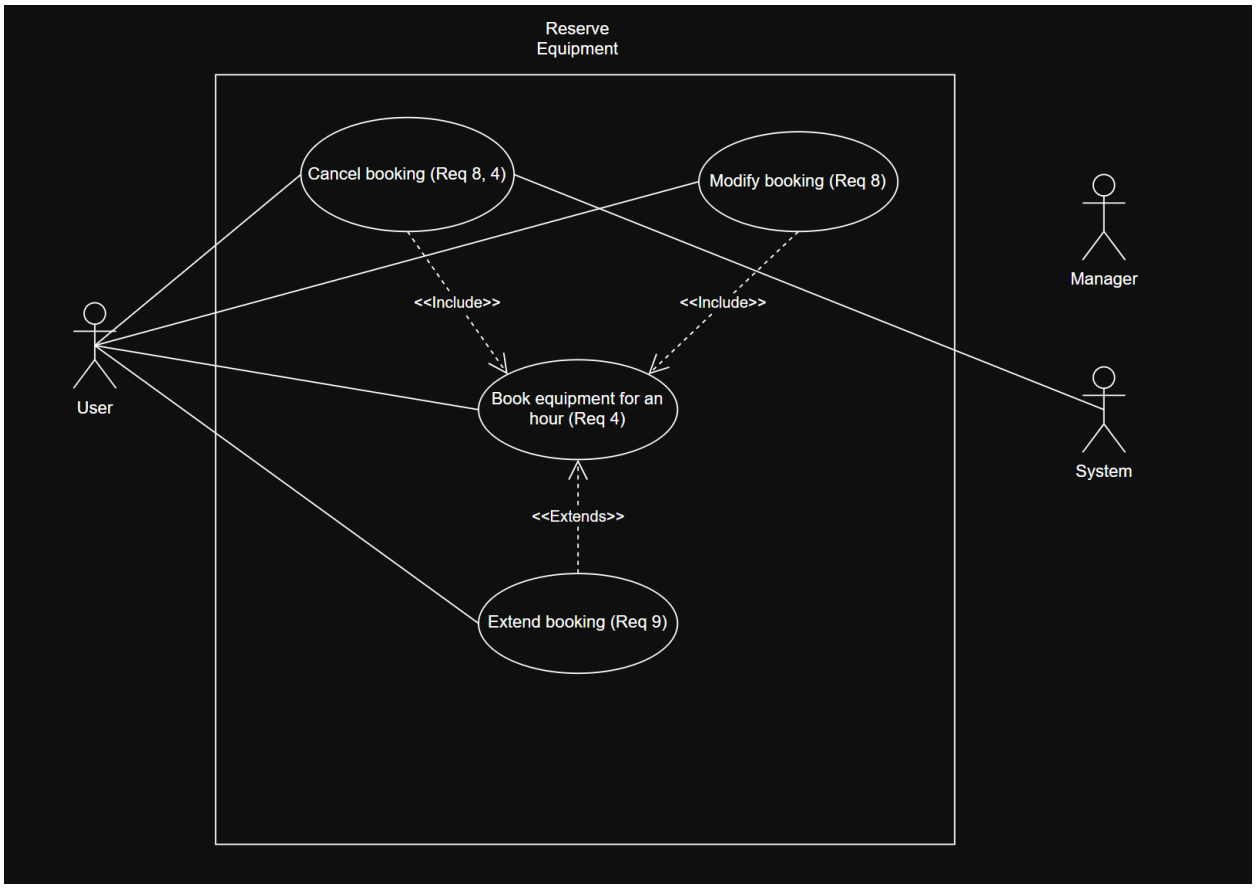
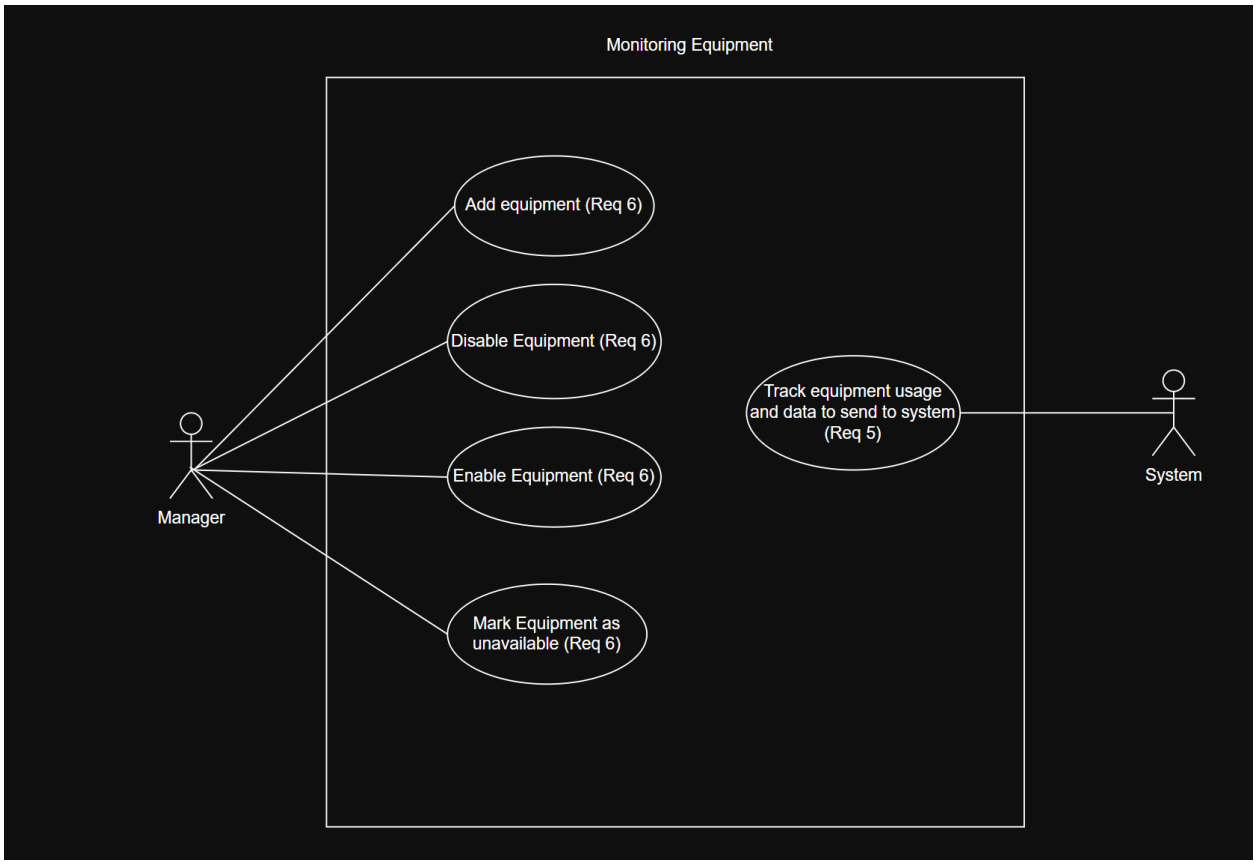

Reserve
Equipment
Cancel booking (Req 8, 4)
Modify booking (Req 8)
<<Include>>
<<Include>>
Book equipment for an
hour (Req 4)
<<Extends>>
Extend booking (Req 9)
User
Manager
System

Diagram 3:


Monitoring Equipment
Add equipment (Req 6)
Disable Equipment (Req 6)
Enable Equipment (Req 6)
Mark Equipment as unavailable (Req 6)
Track equipment usage and data to send to system (Req 5)
Manager
System